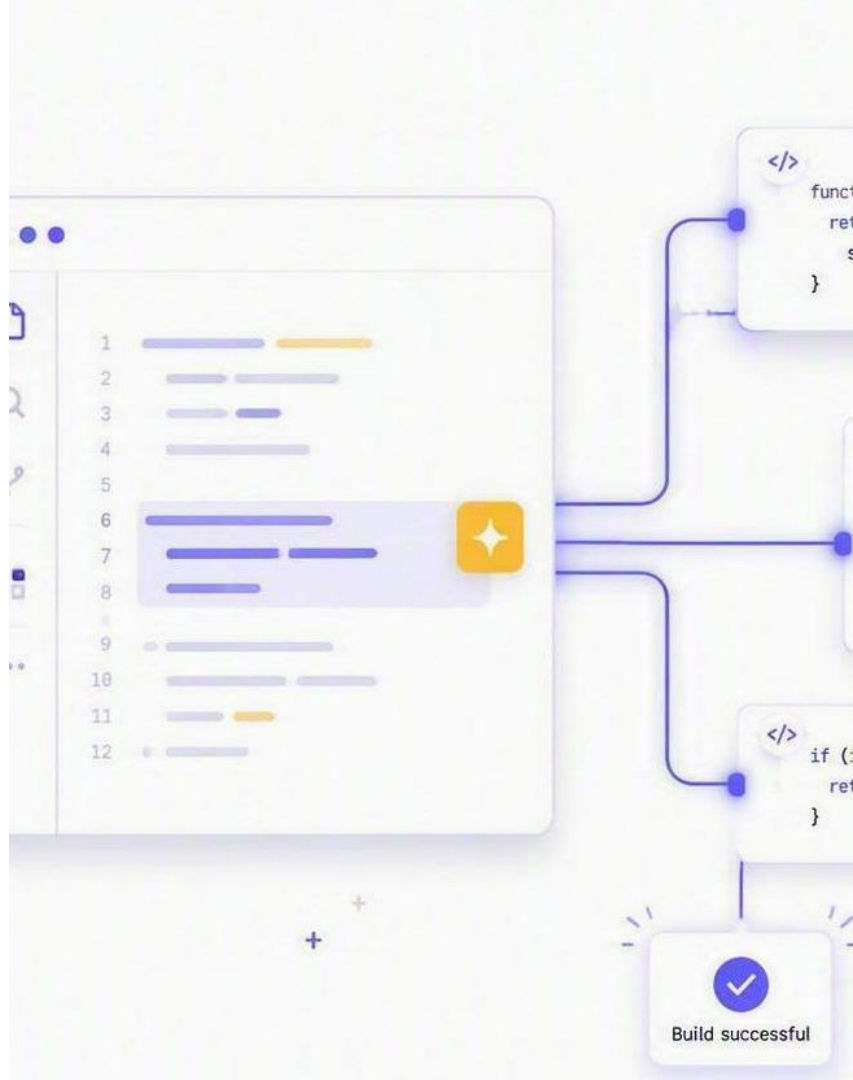


MÓDULO FORMATIVO

GitHub Copilot e ingeniería de solicitudes

Cómo escribir solicitudes eficaces, cómo procesa Copilot cada mensaje y qué papel cumplen los modelos de lenguaje grande.

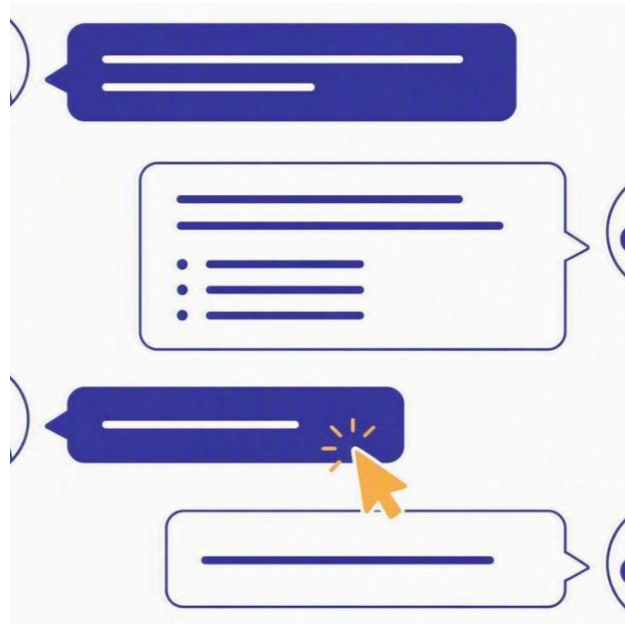
Resumen del módulo



Copilot acelera el flujo de desarrollo

GitHub Copilot, con tecnología de OpenAI, acompaña desde la creación inicial de código hasta implementaciones listas para producción. Comprende el contexto del proyecto a partir de datos de entrenamiento que incluyen lenguaje natural y miles de millones de líneas de código fuente disponible públicamente, incluidos repositorios públicos de GitHub.

- Genera sugerencias contextuales dentro del editor.
- Acelera los cambios de código y las tareas rutinarias.
- Su calidad depende de cómo se formulen las solicitudes.



Qué aborda este módulo

- Principios y buenas prácticas de ingeniería de solicitudes.
- Estrategias avanzadas: solicitud de roles y administración del historial de chat.
- Cómo procesa Copilot las solicitudes para generar respuestas y código.
- Flujo de datos de las sugerencias de código y del chat.
- Rol de los modelos de lenguaje grande (LLM) en GitHub Copilot.
- Creación de solicitudes eficaces para mejorar precisión y relevancia.
- Relación entre las solicitudes y las respuestas de Copilot.
- Gestión de los datos: transmisión segura y filtrado de contenido.

CONCEPTO

Ingeniería de solicitudes

Es el proceso de crear instrucciones claras para guiar sistemas de IA como GitHub Copilot y obtener código adecuado al contexto y a las necesidades específicas del proyecto.

Una solicitud bien diseñada busca obtener código:

Sintácticamente correcto

Compila y respeta las reglas del lenguaje utilizado.

Funcional

Resuelve realmente la tarea planteada en la solicitud.

Adecuado al contexto

Sigue las convenciones y la estructura del proyecto.

Las 4 S de una buena solicitud

Single

Única

Centrar la solicitud en una sola tarea o pregunta bien definida para obtener respuestas precisas.

Specific

Específica

Usar instrucciones explícitas y detalladas para lograr sugerencias más aplicables.

Short

Corta

Mantener la solicitud concisa y directa, equilibrando detalle y claridad.

Surround

Enmarcada

Aportar contexto con nombres de archivo descriptivos y archivos relacionados abiertos.

Buenas prácticas al solicitar

01

Proporcionar claridad

Las solicitudes deben ser claras, específicas y centradas en una única tarea.

02

Proporcionar contexto

La información contextual ayuda a comprender los requisitos. Los comentarios en el código aportan detalles adicionales, y Copilot también usa las pestañas abiertas en paralelo en el editor.

03

Proporcionar ejemplos

Los ejemplos aclaran requisitos y hacen concretos los conceptos abstractos: ayudan a identificar patrones y reducen las iteraciones necesarias.

Ejemplos e iteración

Los ejemplos son especialmente útiles para

- Código reutilizable.
- Plantillas de pruebas.
- Implementaciones repetitivas.
- Características más grandes basadas en patrones existentes.

La primera solicitud no siempre produce código listo para producción: la interacción con Copilot es un diálogo iterativo.

Ciclo de iteración

- 1 Revisar el código sugerido.
- 2 Enriquecer la solicitud con detalles o ejemplos.
- 3 Volver a solicitar una solución.
- 4 Iterar hasta obtener el resultado esperado.

Cada iteración aprovecha la comprensión previa de los requisitos.

EJEMPLOS

Cómo aprende Copilot de las solicitudes

Copilot usa modelos entrenados con grandes cantidades de datos. Los ejemplos orientan al modelo hacia patrones y convenciones específicas.

Sin ejemplos

Genera código a partir de su entrenamiento previo. Adecuado para patrones comunes y funcionalidades estándar.

Con un ejemplo

Produce respuestas más compatibles con el contexto y las convenciones específicas del código.

Con varios ejemplos

Permite implementaciones más sofisticadas, cubriendo distintos escenarios y casos límite.

Mejoran la precisión y reducen el trabajo de revisión manual.

Cadena de mensajes e historial

Las funcionalidades complejas pueden requerir conversaciones extendidas. Cada turno agrega información, pero un historial demasiado largo aumenta el contexto y el procesamiento necesarios.

Ejemplo de conversación progresiva

- Crear una función de autenticación de usuario.
- Agregar control de errores para credenciales no válidas.
- Agregar pruebas unitarias.
- Agregar comentarios JSDoc.
- Optimizar el rendimiento.

2–3 PRU

por turno con avisos largos que arrastran todo el historial, frente a cerca de 1 PRU por solicitud al resumir o restablecer.

Administrar el historial

- Resumir el contexto cuando la conversación se extiende.
- Restablecer la conversación al comenzar una nueva funcionalidad.
- Usar referencias concisas al trabajo anterior.

Solicitud de roles

Consiste en indicar a Copilot que actúe como un tipo específico de experto, lo que puede mejorar la calidad y relevancia de las soluciones en tareas especializadas.

Experto en seguridad

- Saneamiento de entrada.
- Protección ante ataques comunes.
- Validación estándar del sector.
- Buenas prácticas de seguridad.

Optimización del rendimiento

- Algoritmos y estructuras optimizados.
- Uso eficiente de memoria.
- Escalabilidad.
- Supervisión del rendimiento.

Especialista en pruebas

- Cobertura exhaustiva.
- Casos extremos.
- Implementaciones ficticias.
- Pruebas de condiciones de error.

Incorporar conocimiento de dominio desde la primera implementación reduce ciclos de revisión.

Flujo de entrada: de la solicitud al modelo

1

Transmisión segura y contexto

El mensaje viaja por HTTPS. Copilot reúne el código anterior y posterior al cursor, el archivo, las pestañas abiertas, la estructura del proyecto y los lenguajes, y aplica Fill-in-the-Middle.

2

Filtro de proxy

El contexto pasa a un servidor proxy en un inquilino de Microsoft Azure propiedad de GitHub, que filtra el tráfico y bloquea intentos de manipular la solicitud.

3

Filtrado de toxicidad

Antes de generar código se bloquean discursos de odio y contenido inapropiado, y se filtran datos personales como nombres, direcciones o identificadores.

4

Generación con LLM

La solicitud filtrada y analizada se envía a los modelos de lenguaje grande, que generan sugerencias relevantes y adaptadas al proyecto.

Flujo de salida: validación y entrega

5 Procesamiento posterior y validación

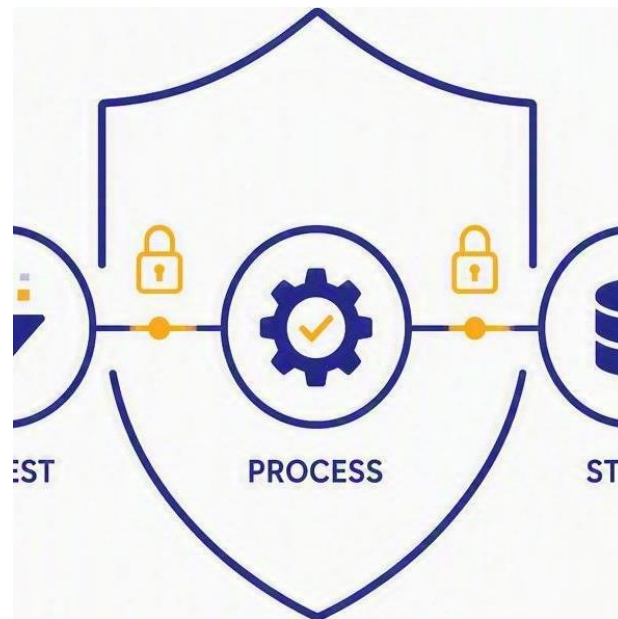
El filtro de toxicidad elimina contenido dañino y el proxy comprueba calidad, seguridad y estándares éticos, buscando errores comunes como XSS o inyección SQL. Opcionalmente, los administradores pueden activar el filtro de código público coincidente, que evita sugerencias de más de unos 150 caracteres muy parecidas a código público. Lo que no supera las comprobaciones se trunca o se descarta.

6 Entrega y bucle de retroalimentación

Solo se entregan las respuestas que superan los filtros. Copilot inicia un bucle de retroalimentación: amplía sus conocimientos con las sugerencias aceptadas y aprende de las modificaciones y los rechazos.

7 Repetición en solicitudes posteriores

El proceso se repite con cada mensaje. Copilot aplica los datos acumulados y el contexto para comprender mejor la intención y refinar la generación de código.



Datos de GitHub Copilot

Copilot controla los datos según el entorno, las características y las configuraciones utilizadas.

Sugerencias de código

- En el editor, Copilot no conserva las solicitudes ni el código u otro contexto para entrenar los modelos fundamentales.
- Las solicitudes se descartan una vez devuelta la sugerencia.
- Los suscriptores individuales pueden optar por no compartir sus solicitudes para el perfeccionamiento del modelo.

Chat de GitHub Copilot

- Formato: resalta fragmentos de código y ofrece integrarlos directamente.
- Interacción: admite preguntas de seguimiento y aclaraciones.
- Historial: conserva la conversación para mantener el contexto.
- Retención: fuera del editor, GitHub conserva normalmente solicitudes, sugerencias y contexto 28 días.
- Igual comportamiento en CLI, Móvil y GitHub.com.

Tipos de solicitud y ventana de contexto

Tipos de solicitud admitidos por el chat

- Preguntas directas sobre conceptos, bibliotecas o solución de errores.
- Solicitudes de código: generar, modificar, corregir o explicar.
- Consultas abiertas sobre procedimientos recomendados o mejoras.
- Solicitudes contextuales con fragmentos de código o escenarios concretos.

Esta variedad permite usar Copilot como un compañero de codificación integral.

Ventanas de contexto limitadas

La ventana de contexto determina cuánto código y texto puede procesar el modelo a la vez.

200–500

líneas de código, o hasta unos miles de tokens

4000

tokens en la ventana actual del chat de Copilot

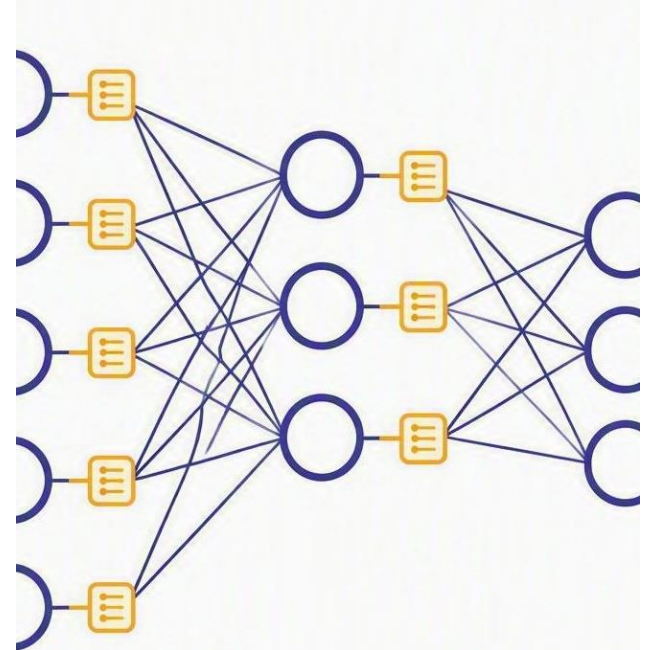
Puede variar según la implementación y versión. Conviene dividir los problemas complejos en consultas más pequeñas y aportar solo los fragmentos relevantes.

Modelos de lenguaje grande (LLM)

Los LLM son modelos de inteligencia artificial diseñados y entrenados para comprender, generar y manipular lenguaje humano.

- Gran volumen de datos de entrenamiento, de orígenes diversos.
- Comprensión contextual para generar texto coherente y relevante.
- Redes neuronales con millones o miles de millones de parámetros.
- Versatilidad para adaptarse a tareas, dominios e idiomas.

En GitHub Copilot, el modelo considera el archivo actual, otros archivos abiertos y las pestañas abiertas en el IDE para generar finalizaciones de código más precisas y pertinentes.



Ajuste de los LLM y optimización LoRA

Ajuste preciso (fine-tuning)

Adapta modelos previamente entrenados a tareas o dominios específicos. Consiste en entrenar el modelo con un conjunto de datos más pequeño y específico de la tarea, el conjunto de datos de destino, aprovechando los conocimientos y parámetros del modelo entrenado con un conjunto de origen más grande. Así se mejora su rendimiento en tareas concretas.

LoRA (Low-Rank Adaptation)

El entrenamiento completo tradicional ajusta todas las partes de la red neuronal: es lento y consume muchos recursos. LoRA adapta modelos preentrenados sin repetir todo el entrenamiento.

- Agrega componentes entrenables más pequeños a cada capa.
- Mantiene sin modificar el modelo original.
- Reduce el tiempo y el consumo de recursos.
- Supera a métodos como los adaptadores y el ajuste de prefijos.

LoRA mejora los LLM para requisitos de codificación con menos componentes entrenables y menos recursos.